Reviewer Pack — Minimal Local Induction Dimension in Algebraic Geometry and …

Entry 0b09e2560d6d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 16:38:57 UTC

Minimal Local Induction Dimension in Algebraic Geometry and Circuit Entanglement Complexity

Entry ID: `0b09e2560d6d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 16:38:57 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Local Induction Dimension) **Field B** (complexity object): Boolean Circuits (Circuit Entanglement Complexity)

Statement:

> For every d-regular graph G, the minimal local induction dimension (mld(G)) of its associated algebraic variety is linearly correlated with its circuit entanglement complexity e(φ_G), such that mld(G) = Θ(e(φ_G)).

Rationale (proposer's reasoning):

> Local induction dimension measures the complexity of an algebraic variety in terms of its inductive definition. If a graph's associated variety has a low local induction dimension, it suggests a simpler structure that could be exploited to simplify circuit entanglement, potentially leading to better lower bounds for circuit complexity.

Taxonomy category: `Local_Induction_Dimension` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `5c67436ecef4b66c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all 30 random seeds, the mean correlation coefficient between mld(G) and e(φ_G) is ≥ 0.8 AND the standard deviation of mld(G) values across seeds is ≤ 3.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "local induction dimension" AND "algebraic geometry" AND "circuit entanglement complexity" - "mld(G)" AND "algebraic variety" AND "θ(e(φ_G))" - "Boolean circuits" AND "minimal local induction dimension" AND "linear correlation"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.0s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if (n * d) % 2 != 0:
            return None
        graph = {i: [] for i in range(n)}
        edges_added = set()
        for _ in range(d * n // 2):
            while True:
                u, v = random.sample(range(n), 2)
                if (u, v) not in edges_added and (v, u) not in edges_added:
                    graph[u].append(v)
                    graph[v].append(u)
                    edges_added.add((u, v))
                    break
        return graph

    def gaussian_elimination(matrix):
        m, n = len(matrix), len(matrix[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                    max_row = j
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
            factor = Fraction(matrix[i][i])
            for j in range(n):
                matrix[i][j] /= factor
```

```
        for j in range(m):
            if i != j:
                factor = Fraction(matrix[j][i])
                for k in range(n):
                    matrix[j][k] -= factor * matrix[i][k]
    return matrix

def determinant(matrix):
    m, n = len(matrix), len(matrix[0])
    if m != n:
        raise ValueError("Matrix must be square")
    if m == 1:
        return matrix[0][0]
    det = Fraction(0)
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
        det += (-1)**j * matrix[0][j] * determinant(submatrix)
    return det

def circuit_entanglement_complexity(graph, n):
    # Placeholder function to simulate entanglement complexity
    # Replace with actual computation if available
    return random.random() * n

def minimal_local_induction_dimension(graph, n):
    # Placeholder function to simulate local induction dimension
    # Replace with actual computation if available
    return random.random() * n

n = 30
d = 3
graph = generate_d_regular_graph(n, d)
if not graph:
    return {
        "metric_name": "mld(G)",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

mld_G = minimal_local_induction_dimension(graph, n)
e_phi_G = [circuit_entanglement_complexity(graph, n) for _ in range(n)]

if len(e_phi_G) == 0:
    return {
        "metric_name": "mld(G)",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "empty_e_phi_G"
    }

x_bar = sum(mld_G for _ in range(n)) / n
```

```
    y_bar = sum(e_phi_G) / len(e_phi_G)

    numerator = sum((mld_G - x_bar) * (e_phi_G[i] - y_bar) for i, _ in enumerate(e_phi_G))
    denominator = math.sqrt(sum((mld_G - x_bar)**2 for _ in range(n))) * math.sqrt(sum((y_bar - y_

    if denominator == 0:
        correlation_coefficient = None
    else:
        correlation_coefficient = numerator / denominator

    return {
        "metric_name": "mld(G)",
        "metric_value": correlation_coefficient,
        "instances_tested": n,
        "n_max": n,
        "conjecture_holds": False if correlation_coefficient is None else abs(correlation_coeffici
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_dev = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_valu
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fracti
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fracti
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"unsupported\" first_failing_seed={first_failing
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
lue': None, 'instances_tested': 30, 'n_max': 30, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'mld(G)', 'metric_value': None, 'instances_tested': 30, 'n_max': 30, 'conjectu
TRIAL: {'metric_name': 'mld(G)', 'metric_value': None, 'instances_tested': 30, 'n_max': 30, 'conjectu
TRIAL: {'metric_name': 'mld(G)', 'metric_value': None, 'instances_tested': 30, 'n_max': 30, 'conjectu
TRIAL: {'metric_name': 'mld(G)', 'metric_value': None, 'instances_tested': 30, 'n_max': 30, 'conjectu
TRIAL: {'metric_name': 'mld(G)', 'metric_value': None, 'instances_tested': 30, 'n_max': 30, 'conjectu
TRIAL: {'metric_name': 'mld(G)', 'metric_value': None, 'instances_tested': 30, 'n_max': 30, 'conjectu
TRIAL: {'metric_name': 'mld(G)', 'metric_value': None, 'instances_tested': 30, 'n_max': 30, 'conjectu
TRIAL: {'metric_name': 'mld(G)', 'metric_value': None, 'instances_tested': 30, 'n_max': 30, 'conjectu
```

```
TRIAL: {'metric_name': 'mld(G)', 'metric_value': None, 'instances_tested': 30, 'n_max': 30, 'conjectu
TRIAL: {'metric_name': 'mld(G)', 'metric_value': None, 'instances_tested': 30, 'n_max': 30, 'conjectu
TRIAL: {'metric_name': 'mld(G)', 'metric_value': None, 'instances_tested': 30, 'n_max': 30, 'conjectu
TRIAL: {'metric_name': 'mld(G)', 'metric_value': None, 'instances_tested': 30, 'n_max': 30, 'conjectu
TRIAL: {'metric_name': 'mld(G)', 'metric_value': None, 'instances_tested': 30, 'n_max': 30, 'conjectu
RESULT: FALSIFIED counterexample="unsupported" first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> The test code uses placeholder functions for both 'circuit_entanglement_complexity' and 'minimal_local_induction_dimension', which are not implemented according to the conjecture's mathematical definitions. This makes it impossible to assess whether the conjecture holds, as the actual computation of these metrics is missing.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test code failed to compute the actual values for both 'circuit_entanglement_complexity' and 'minimal_local_induction_dimension', which are requir | next: Implement the actual computation for both 'circuit_entanglement_complexity' and 'minimal_local_induction_dimension' according to their mathematical definitions and retest the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13360
2	propose	ollama_remote	glm4:latest	0	0	17777
3	propose	ollama_remote	glm4:latest	0	0	15052
4	preregistration	ollama_remote	glm4:latest	0	0	9087
5	novelty	ollama_remote	glm4:latest	0	0	8563
6	novelty	ollama_remote	glm4:latest	0	0	8872
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16316
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11633
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10815
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15593
11	critic	ollama_remote	glm4:latest	0	0	26699
12	judge	ollama_remote	glm4:latest	0	0	9523

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 163292 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/0b09e2560d6d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0b09e2560d6d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0b09e2560d6d.tar.gz` (if generated)